

Action Memory for Smooth Control: Preventing Bang-Bang Behavior with Differentiable Zero-Mean Constraints

Implementation Report
CaNS Opposition Control with MADDPG

February 4, 2026

Abstract

This report addresses two interrelated problems in multi-agent reinforcement learning for turbulent flow control: (1) bang-bang oscillations caused by lack of temporal context, and (2) gradient mismatch from non-differentiable zero-mean constraints. We show that action memory (state augmentation with previous actions) combined with differentiable zero-mean correction enables smooth, physically valid control while maintaining policy trainability. The solution is conceptually simple but requires careful implementation to preserve gradient flow while guaranteeing exact mass conservation.

Contents

1	Problem Statement	3
1.1	The Bang-Bang Control Problem	3
1.2	The Zero-Mean Constraint Problem	3
2	Unified Solution: Action Memory + Differentiable Constraints	3
2.1	State Augmentation with Previous Action	3
2.2	Differentiable Zero-Mean Correction	3
2.2.1	Training Forward Pass	3
2.2.2	Inference (No Noise)	4
3	How Trainability and Enforcement Coexist	4
3.1	The Seeming Paradox	4
3.2	Gradient Flow Explanation	4
3.2.1	Without Differentiable Correction (Post-Processing)	4
3.2.2	With Differentiable Correction	4
3.3	Learning Dynamics	5
3.4	Why This Doesn't Break Learning	5
4	Critical Implementation Details	5
4.1	Noise and Clipping Interaction	5
4.2	Global vs Per-Tile Zero-Mean	6
5	Results and Validation	6
5.1	Expected Training Behavior	6
5.2	Convergence Indicators	6
5.3	Action Memory Impact	7

6	Summary: Key Insights	7
7	Code Implementation	7
7.1	Training Script Pattern	7
7.2	Loss Components	8
8	Conclusion	8

1 Problem Statement

1.1 The Bang-Bang Control Problem

Multi-agent RL policies for flow control exhibit high-frequency oscillations (bang-bang behavior) characterized by:

- Rapid switching between extreme actions: $a_t \in \{-1, +1\}$
- High temporal gradient: $\|\nabla_t a\|_{\text{RMS}} \approx 0.4$
- Good drag reduction but poor physical realism

Root cause: The policy $\pi(\mathbf{u}_t, \mathbf{w}_t)$ creates a closed-loop feedback instability:

$$\text{Bang-bang } a_t \rightarrow \text{Flow waves} \rightarrow \text{Oscillating } (\mathbf{u}_{t+1}, \mathbf{w}_{t+1}) \rightarrow \text{Bang-bang } a_{t+1} \quad (1)$$

Without temporal context, the policy cannot detect or dampen this self-reinforcing cycle.

1.2 The Zero-Mean Constraint Problem

Flow control requires mass conservation: $\sum_{i,j} a_{ij} = 0$. Standard implementations apply this as **post-processing**:

$$a^{\text{exec}} = a^{\text{raw}} - \text{mean}(a^{\text{raw}}) \quad (2)$$

Critical issues:

1. **Gradient mismatch:** Backpropagation doesn't see the correction, policy assumes direct control
2. **Induced oscillations:** Varying mean μ_t creates oscillating corrections even for smooth a_t^{raw}
3. **Credit assignment failure:** Policy never learns to output near-zero-mean actions

2 Unified Solution: Action Memory + Differentiable Constraints

2.1 State Augmentation with Previous Action

Modified state representation:

$$\mathbf{s}_t = [\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1}] \in \mathbb{R}^{3 \times 8 \times 8} \quad (3)$$

Benefits:

- Policy sees full feedback loop: $\pi(\mathbf{u}_t, \mathbf{w}_t, \mathbf{a}_{t-1})$ learns "I was oscillating, dampen response"
- Enables temporal consistency without explicit penalties
- Maintains Markov property (history summarized in $\mathbf{a}_{t-1}$)

2.2 Differentiable Zero-Mean Correction

Key insight: Make the zero-mean correction part of the computational graph during training.

2.2.1 Training Forward Pass

Algorithm 1 Differentiable Zero-Mean Correction (Training)

- 1: **Forward pass:** $\mathbf{A}^{\text{raw}} = \pi_{\theta}(\mathbf{S})$ [$\in [-1, 1]$ from tanh]
- 2: **Zero-mean correction (differentiable):**

$$\mu = \text{mean}(\mathbf{A}^{\text{raw}}), \quad \mathbf{A}^{\text{zm}} = \mathbf{A}^{\text{raw}} - \mu$$

- 3: **Add exploration noise:** $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$, $\epsilon \leftarrow \epsilon - \text{mean}(\epsilon)$
 - 4: **Add noise:** $\mathbf{A}^{\text{noisy}} = \mathbf{A}^{\text{zm}} + \epsilon$
 - 5: **Clip to bounds:** $\mathbf{A}^{\text{clip}} = \text{clip}(\mathbf{A}^{\text{noisy}}, -1, 1)$ [breaks zero-mean!]
 - 6: **Final zero-mean guarantee:** $\mathbf{A} = \mathbf{A}^{\text{clip}} - \text{mean}(\mathbf{A}^{\text{clip}})$
 - 7: **Compute loss:** $\mathcal{L} = -Q(\mathbf{S}, \mathbf{A}) + \lambda_s \mathcal{L}_{\text{spatial}} + \lambda_g \mathcal{L}_{\text{global_mean}}$
 - 8: **Backpropagate:** $\nabla_{\theta} \mathcal{L}$ [gradients flow through all corrections]
-

Algorithm 2 Inference with Exact Zero-Mean

- 1: **Forward pass:** $\mathbf{A}^{\text{raw}} = \pi_{\theta}(\mathbf{S})$ [$\in [-1, 1]$ from tanh]
 - 2: **Zero-mean guarantee:** $\mathbf{A} = \mathbf{A}^{\text{raw}} - \text{mean}(\mathbf{A}^{\text{raw}})$
 - 3: **Execute:** $\mathbf{A}$ is exactly zero-mean, no clipping needed
-

2.2.2 Inference (No Noise)

3 How Trainability and Enforcement Coexist

3.1 The Seeming Paradox

Question: If we always apply $a = a^{\text{raw}} - \text{mean}(a^{\text{raw}})$, how does the policy learn anything? Won't it just output arbitrary actions since they'll be corrected anyway?

Answer: Differentiability makes all the difference.

3.2 Gradient Flow Explanation

Consider a single training step with 2 actions for simplicity: $a^{\text{raw}} = [a_1, a_2]$.

3.2.1 Without Differentiable Correction (Post-Processing)

$$\text{Forward: } a^{\text{raw}} = [0.5, -0.3] \Rightarrow \mu = 0.1 \quad (4)$$

$$a^{\text{exec}} = [0.5 - 0.1, -0.3 - 0.1] = [0.4, -0.4] \quad (5)$$

$$\text{Gradient: } \frac{\partial \mathcal{L}}{\partial a_1} = -2.0 \quad (\text{from critic feedback}) \quad (6)$$

$$\text{Update: } \theta \leftarrow \theta + \eta \cdot 2.0 \quad (\text{policy doesn't see correction!}) \quad (7)$$

Policy learns as if it directly controls $[0.5, -0.3]$, but environment saw $[0.4, -0.4]$. **Mismatch!**

3.2.2 With Differentiable Correction

$$\text{Forward: } a^{\text{raw}} = [0.5, -0.3], \quad \mu = \text{mean}([0.5, -0.3]) = 0.1 \quad (8)$$

$$a = a^{\text{raw}} - \mu = [0.4, -0.4] \quad (9)$$

$$\text{Gradient chain rule: } \frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial a^{\text{raw}}} \cdot \frac{\partial a^{\text{raw}}}{\partial \theta} \quad (10)$$

The Jacobian captures coupling:

$$\frac{\partial a_i}{\partial a_j^{\text{raw}}} = \begin{cases} 1 - \frac{1}{N} & i = j \\ -\frac{1}{N} & i \neq j \end{cases} \quad (11)$$

Key effect: If a_1 increases, a_2 decreases (to maintain zero-mean). Policy learns this coupling!

3.3 Learning Dynamics

Early training:

- Policy outputs arbitrary a^{raw} with large $|\mu|$ (e.g., $\mu = 0.5$)
- Large corrections applied: $a = a^{\text{raw}} - 0.5$
- Gradients penalize large mean through:

$$\mathcal{L}_{\text{global_mean}} = \lambda_g (\text{mean}(a^{\text{raw}}))^2 \quad (12)$$

Late training:

- Policy learns to output $a^{\text{raw}} \approx \text{zero-mean}$
- Small corrections: $|\mu| < 0.05$
- Actions are **inherently near-zero-mean**, not arbitrarily corrected
- Final correction is just a safety guarantee

3.4 Why This Doesn't Break Learning

Intuition: The constraint $\sum a_i = 0$ removes **one degree of freedom** (the global offset). But we have $N = 64 \times 64 = 4096$ actions. The policy still has 4095 degrees of freedom to optimize!

Analogy: Imagine training a robot arm to reach targets while constrained to a table surface. The vertical position is always $z = 0$ (enforced). Does this prevent learning? No! The robot learns to position (x, y) optimally. Similarly, the policy learns the *pattern* of actions (spatial structure) while the global mean is constrained to zero.

4 Critical Implementation Details

4.1 Noise and Clipping Interaction

Problem: Clipping breaks zero-mean!

$$a^{\text{noisy}} = [1.2, -0.8, 0.1, -0.5] \quad (\text{mean} = 0) \quad (13)$$

$$a^{\text{clip}} = [1.0, -0.8, 0.1, -0.5] \quad (\text{mean} = -0.05 \neq 0) \quad (14)$$

Solution: Always re-apply zero-mean after clipping:

$$a^{\text{final}} = a^{\text{clip}} - \text{mean}(a^{\text{clip}}) \quad (15)$$

Optimization: When noise is annealed to zero ($\sigma = 0$ in late training), skip clipping entirely since $\tanh$ already guarantees $a \in [-1, 1]$.

4.2 Global vs Per-Tile Zero-Mean

Invalid approach: Forcing each 8×8 agent tile to have zero-mean:

$$\mathcal{L}_{\text{per-tile}} = \sum_{i=1}^{64} (\text{mean}(\mathbf{a}_i))^2 \quad \text{WRONG!} \quad (16)$$

Why invalid: Different flow regions require different average actuation:

- Upstream: net blowing ($\text{mean} > 0$)
- Downstream: net suction ($\text{mean} < 0$)

Correct approach: Global zero-mean only:

$$\mathcal{L}_{\text{global}} = \left(\frac{1}{64 \times 64} \sum_{i,j} a_{ij} \right)^2 \quad (17)$$

This respects physics (mass conservation) without artificial constraints.

5 Results and Validation

5.1 Expected Training Behavior

Episode 1-3 (Buffer warmup):

- Random exploration, filling replay buffer
- No training yet (`training_step` = 0)
- Only basic metrics logged

Episode 4+ (Training active):

- `global_mean_loss` decreases: Policy learning constraint
- `action_mean` approaches zero: $|\mu| < 0.05$
- `actor_grad_norm` stable: < 10
- `dpx_mean` decreases: Drag reduction improving

5.2 Convergence Indicators

Metric	Convergence Target	Meaning
<code>global_mean_loss</code>	< 0.01	Policy outputs near-zero mean
<code>action_mean</code>	< 0.05	Small corrections needed
<code>actor_exploding</code>	$= 0$	Stable training
<code>temporal_gradient_rms</code>	< 0.2	Smooth actions

5.3 Action Memory Impact

Without action memory:

- Temporal gradient RMS: ≈ 0.4
- FFT shows high-frequency peaks
- Bang-bang oscillations persist

With action memory:

- Temporal gradient RMS: < 0.2
- Smooth frequency spectrum
- Adaptive temporal consistency

6 Summary: Key Insights

1. **Action memory breaks feedback loops:** Seeing $\mathbf{a}_{t-1}$ lets policy detect and dampen oscillations
2. **Differentiability $\neq$ no constraint:** Making the correction differentiable allows gradient flow while still enforcing exact zero-mean
3. **Policy learns the constraint:** Through gradient coupling, network learns to output near-zero-mean actions, minimizing correction magnitude
4. **Safety + trainability:** Differentiable correction during training + final guarantee ensures both learning and physical validity
5. **Global constraint only:** Per-tile zero-mean is physically invalid; only global mass conservation matters
6. **Clipping requires re-correction:** Always reapply zero-mean after clipping to guarantee exact constraint satisfaction

7 Code Implementation

7.1 Training Script Pattern

```
# Forward pass through all agents
actions_raw = maddpg.act(observations) # tanh \in [-1, 1]

# Differentiable zero-mean correction
global_mean = actions_raw.mean()
actions = actions_raw - global_mean # Gradients flow!

# Add zero-mean exploration noise (if training)
if noise_scale > 0:
    noise = np.random.normal(0, noise_scale, actions.shape)
    noise = noise - noise.mean() # Zero-mean noise
    actions = actions + noise
    actions = np.clip(actions, -1, 1) # May break zero-mean
```

```

# CRITICAL: Re-apply zero-mean after clipping
actions = actions - actions.mean()

# actions is now exactly zero-mean, ready for environment
next_observations, rewards, dones, infos = env.step(actions)

# Update networks with gradient flow through zero-mean correction
maddpg.update_batched(batch)

```

7.2 Loss Components

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}[Q(\mathbf{s}, \mathbf{a})] + \lambda_s \mathcal{L}_{\text{spatial}} + \lambda_g \mathcal{L}_{\text{global_mean}} + \lambda_c \mathcal{L}_{\text{consistency}} \quad (18)$$

$$\mathcal{L}_{\text{spatial}} = \mathbb{E} \left[\|\nabla_x \mathbf{a}\|^2 + \|\nabla_y \mathbf{a}\|^2 \right] \quad (19)$$

$$\mathcal{L}_{\text{global_mean}} = (\text{mean}(\mathbf{a}))^2 \quad (20)$$

$$\mathcal{L}_{\text{consistency}} = \mathbb{E} \left[\sum_{\text{similar pairs}} \max(0, \|\mathbf{a}_i - \mathbf{a}_j\| - m) \right] \quad (21)$$

Recommended hyperparameters:

- $\lambda_s = 0.5$ (spatial smoothness)
- $\lambda_g = 0.05$ (global zero-mean)
- $\lambda_c = 0.0$ (consistency - often not needed with action memory)

8 Conclusion

The combination of action memory and differentiable zero-mean correction solves two problems simultaneously:

1. **Bang-bang oscillations:** Action memory provides temporal context to detect and prevent feedback instability
2. **Constraint learning:** Differentiable correction enables policy to learn the zero-mean structure rather than fighting against it

The key conceptual insight is that **differentiability enables learning of the constraint**, not removal of the constraint. The policy learns to operate within the zero-mean manifold, making corrections progressively smaller while still maintaining exact mass conservation for solver stability.